

Data Structure

Level 1

Big $O(n^2)$: Quadratic
Time Function

Data Structure

Level 1

Complexity of an Algorithm

Big O

$O(n^2)$

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, adding a human element to the technical theme.

Data Structure

Level 1

Quick Review

Big O for the following Algorithms:

Algorithm 1

```
char GetLastCharacter(string S1)
{
    return S1[ S1.length() - 1 ];
}
```

O(1)

Algorithm 2

```
char GetLastCharacter2(string S1)
{
    int n = S1.length() - 1 ;

    for ( int i = 0 ; i <= n ; i++ )
    {
        if ( i == n )
        {
            return S1 [ n ] ;
        }
    }
}
```

O(n)

Data Structure

Level 1

Calculating Algorithm Complexity $O(n^2)$

Calculating Algorithm Complexity:

```
int MultiplicationSum(short n)
{
```

Number of Steps outside loop1 = 4

Number of Steps inside loop2 = 5

```
    int Sum = 0;
```

```
    for (int i = 1; i <= n; i++)
    {
```

```
        for (int j = 1; j <= n; j++)
        {
```

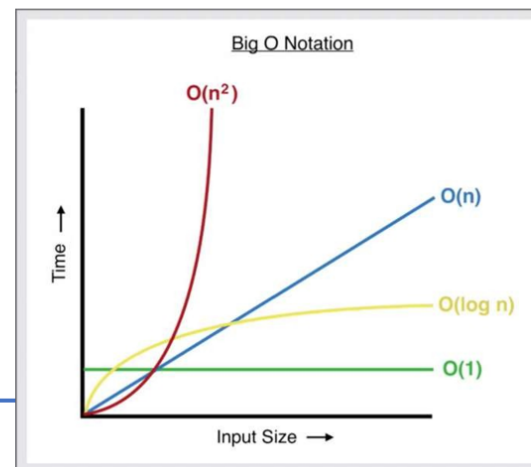
```
            Sum = Sum + (i * j);
        }
```

Big O =
 $5 * n \rightarrow n$

Big O =
 $n * (3 + n)$
 $\rightarrow 3n + n^2$
 $\rightarrow n^2$

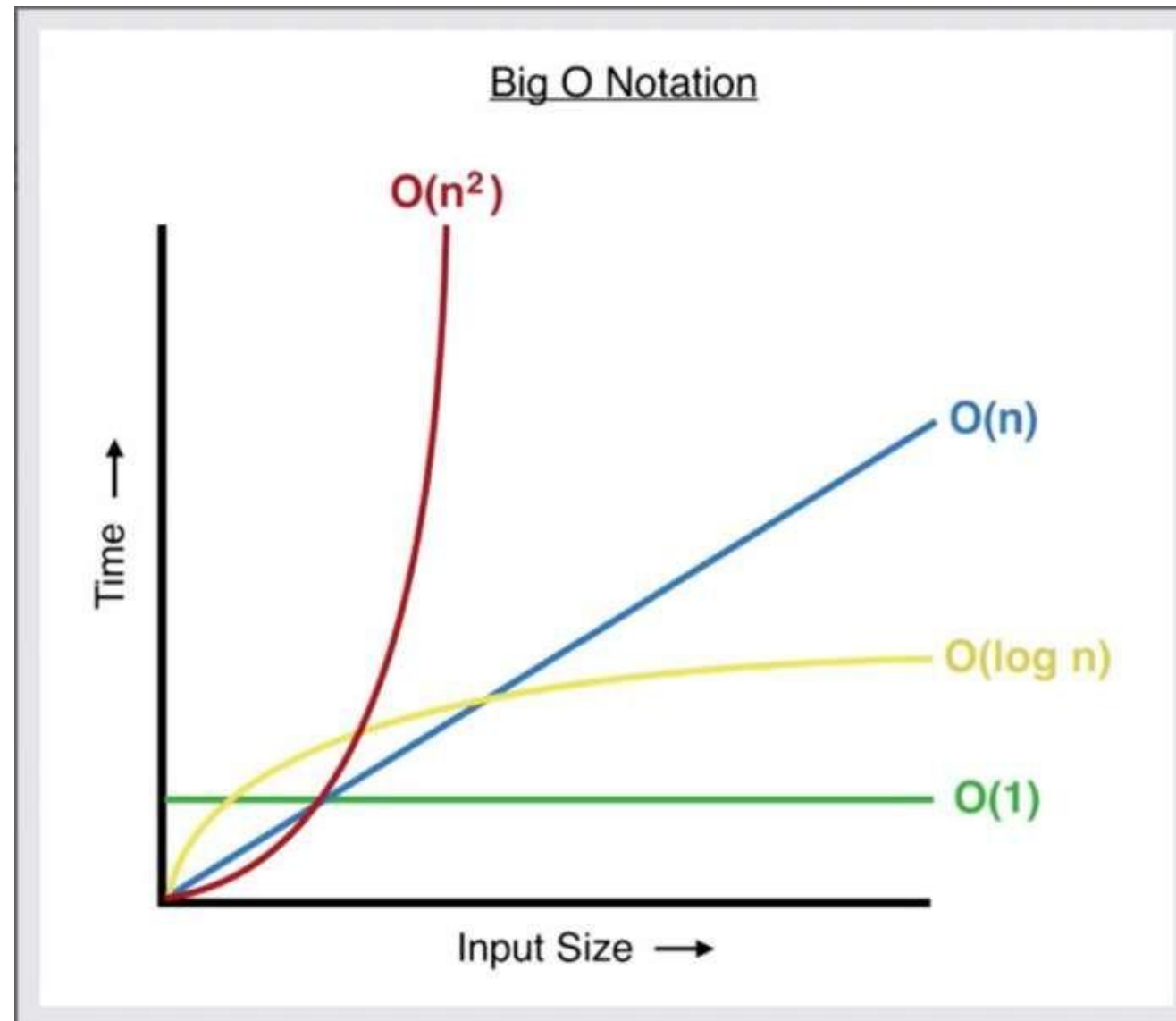
Big O =
 $4 + n^2 \rightarrow n^2$

```
    return Sum;
}
```



Depends on n size, and relation is Quadratic Time Function.

Which is faster?



Thank you 😊